

OBJECT ORIENTED SOFTWARE DESIGN PROJECT

ITERATION 1

Requirement Analysis Document (RAD)

GROUP 17	
Name-Surname	No
Berker Öner	150122018
Miray Piyade	150122026
Ceyhun Erer	150123070
Elifsena Ünver	150123063
Muhammed Emin Erdoğan	150123048

Project Description

The **RAG (Retrieval-Augmented Generation) System** is an intelligent question-answering pipeline designed to provide accurate and contextually relevant answers to user queries by leveraging a structured document corpus. It combines information retrieval, natural language understanding, and answer generation in a modular architecture.

The system operates as a multi-stage pipeline: it first detects the intent of the question, then generates a set of search terms, retrieves relevant documents, reranks the results based on relevance and configurable scoring criteria, and finally generates a textual answer while validating its citations. If no answer can be confidently produced, a fallback mechanism ensures a default response.

Key features of the system include:

- **Intent Detection:** Identifies the type of user question to guide search and answer strategies.
- **Query Writing:** Converts the user question and detected intent into effective search terms.
- **Document Retrieval:** Retrieves relevant passages from a preprocessed document store.
- **Reranking:** Orders retrieved passages based on configurable relevance metrics, including source priority, title matches, and term proximity.
- **Answer Generation:** Produces a final answer with validated citations, ensuring reliability.
- **Trace Logging:** Captures detailed logs of each stage execution for debugging and monitoring.
- **Modularity & Extensibility:** The system is designed for easy integration of new intents, retrievers, or reranking strategies.

The system is implemented in Java and provides a Command-Line Interface (CLI) for user interaction, ensuring portability and ease of use. Its design emphasizes maintainability, performance, and scalability to handle large document collections effectively.

Glossary

Core Classes / Pipeline

- **RagCli**
 - Entry point of the CLI application
 - Receives the user's question and executes the RAG pipeline.
- **RagOrchestrator**
 - GRASP Controller.
 - Builds the pipeline, creates the context, manages trace logs, and executes the pipeline
- **DefaultPipeline**
 - Executes pipeline stages (PipelineStage) in order.
 - Publishes trace logs after each stage.
- **PipelineStage (interface)**
 - Base interface for all pipeline stages.
 - Methods:
 - `getName()` → returns the stage name (used for trace logs)
 - `run(Context, TraceBus)` → executes the stage
- **IntentDetectionStage**
 - Detects the intent from the question (uses IntentDetector).
 - Stores result into Context.intent.
- **QueryWritingStage**
 - Generates search terms based on the question and the detected intent (uses QueryWriter).
- **RetrievalStage**
 - Uses the Retriever to collect hits from documents using the generated terms.
- **RerankingStage**
 - Reorders the hits after retrieval (uses Reranker).
- **AnswerStage**
 - Generates the final answer using AnswerAgent.generateAnswer(hits, query).
 - Validates citations via CitationValidator.
- **FinalizeStage**
 - If the answer is empty, produces a fallback response using FallbackHandler

Configuration

- **Config**
 - Holds pipeline parameters.
 - Examples: paths (intents.yaml, stopwords.yaml, docs.json, reranker.yaml), topK, sourcePriority, logDir.
 - defaultConfig() → creates default values.
 - withQuestion(String) → derives a new config from the existing one with a new question.
- **ConfigLoader**
 - Loads .json or simplified .yaml/.yml config files.
 - Populates strategy selection fields: retrieverType, rerankerType
 - Handles fallback to defaults.

Intent & NLP

- **Intent (enum)**
 - Question categories: StaffLookup, CourseInfo, Policy, Unknown.
- **IntentDetector (interface)**
 - Detects the intent based on the input question.
- **RuleIntentDetector**
 - Rule-based intent detection.
 - Loads rules from YAML/JSON and checks matches using priority.
- **TextNormalizer**
 - Normalizes text and performs tokenization.
- **HeuristicQueryWriter**
 - Creates a search query (terms) using stopwords and intent-based boosting.
- **QueryWriter (interface)**
 - write(String question, Intent intent) → returns a list of search terms

Retrieval

- **Document**
 - Represents a single document: id, source, title, text.
- **DocumentStore**
 - Loads and stores documents from a JSON file.
- **Retriever (interface)**
 - retrieve(question, terms, documents) → List<Hit>
- **KeywordRetriever**
 - Basic keyword-based retrieval.

- **Hit (record)**
 - Represents a retrieval result: document section + score info.

Reranking

- **Reranker (interface)**
 - rerank(List<String> terms, List<Hit> hits, Object config)
- **NoOpReranker**
 - Does nothing; returns hits as they are.
- **SimpleReranker**
 - Recalculates scores using proximity, title boosting, and source boosting.

Answer & Fallback

- **Answer**
 - Pipeline output: text + citations.
- **AnswerAgent**
 - generateAnswer(List<Hit> hits, String query)
 - Answer depends on both the hits and original query.
- **TemplateAnswerAgent**
 - Performs regex-based sentence splitting avoiding abbreviations
 - Scores sentences by query-term matching
 - Selects the most relevant sentence
 - Builds citation in format: docId:section:start-end
 - Ensures proper sentence finalization
- **CitationValidator**
 - Verifies the validity of sources in the answer.
- **FallbackHandler**
 - Produces a default/placeholder answer when needed.

Tracing / Logging

- **TraceBus**
 - Collects TraceEvent objects and sends them to sinks.
- **TraceSink (interface)**
 - Receives TraceEvent via accept(TraceEvent).
- **ConsoleTraceSink**
 - Writes trace events to the console. Writes trace events to the console.
- **JsonlTraceSink**
 - Writes trace events in JSONL format to a file.

- **TraceEvent**
 - Contains stage name, summary, duration, error info, and timestamp.

Utilities

- **JsonUtils**
 - Minimal JSON parser
 - Methods: parse, expectObject, expectArray, expectString, expectNumber
- **FileUtils**
 - File operations (TODO)
- **TimerUtils**
 - now() → returns current time millis

Functional and Non-Functional Requirements

Functional Requirements

1. Question Input

- The system must accept free-form natural language questions from the user via CLI (RagCli).
- The system must allow providing the question either: directly via CLI (--q), or interactively through prompt input.
- If no question is provided, the system must request one from the user.

2. Configuration Management

- The system must load configuration settings (file paths, strategies, topK, source priority) from a JSON or YAML configuration file (ConfigLoader)
- The system must allow overriding the reranker type from CLI (--reranker).
- The system must create an effective runtime configuration using the base config and user overrides.
- Default configuration values must be provided (Config.defaultConfig()).

3. Intent Detection

- The system must determine the user's intent (StaffLookup, CourseInfo, Policy, Unknown) using a rule-based mechanism (RuleIntentDetector).
- Intent must be stored in the Context.

4. Query Writing

- The system must generate optimized search terms using QueryWriter.
- The system must: tokenize the user question, remove stopwords, include intent-

specific boosters, normalize text.

- Generated terms must be saved in Context.

5. Document Retrieval

- The system must load documents from a JSON file using DocumentStore.
- The system must retrieve relevant texts using Retriever.retrieve(question, terms, documents).
- A list of Hit objects must be produced and stored in the Context.
- KeywordRetriever must score matches based on: direct question containment, term matching, source priority.

6. Reranking

- The system must reorder hits using Reranker.
- The system must support multiple reranking strategies:
NoOpReranker
SimpleReranker
- SimpleReranker must adjust scores using: source boosts, title boosts, proximity bonuses.
- Reranked hits must be saved in Context.

7. Answer Generation

- The system must use AnswerAgent.generateAnswer(hits, query) to create the final answer.
- The system must:
Select the most relevant sentence based on query-term similarity
Produce a well-formed answer sentence
Construct citation ranges in the format: docId:section:start-end
- Generated answer must be stored in Context.

8. Citation Validation

- Generated answers must have their citations validated (CitationValidator).
- Invalid citations must prevent the answer from being accepted and must trigger error handling.

9. Fallback Mechanism

- If intent is unknown, no hits are found, or answer generation fails: The system must produce a fallback answer via FallbackHandler.
- The fallback reason must be stored in Context.

10. Pipeline Execution

- Invalid The system must execute stages sequentially through DefaultPipeline.
- The system must stop execution and propagate errors when a stage fails.
- Each stage must update the shared Context.

11. Trace Logging

- The system must record a TraceEvent after each stage.
- Every TraceEvent must include: stage name, summary, duration, error
- The system must log trace events to: console (ConsoleTraceSink), JSONL file (JsonlTraceSink)

12. Output Delivery

- The system must print the final answer text to the CLI.
- The system must also print the list of citations if available.
- If answer generation fails, the fallback answer must be printed.

Non-Functional Requirements

1. Performance

- The pipeline must execute efficiently regardless of document set size.
- Response times must remain within reasonable limits for typical user queries.

2. Scalability

- The system must support increasing document volumes without performance degradation.
- New intents or terms must be adaptable without requiring major architectural redesign.

3. Reliability

- Pipeline stages must handle exceptions and errors gracefully.
- A fallback mechanism must trigger automatically if a specific stage fails.

4. Maintainability

- The codebase must be modular, with strictly independent stages.
- Updates or bug fixes in one module should not cascade to others.

5. Usability

- Users must be able to interact easily via the CLI (RagCli).
- Inputting questions and receiving answers must be straightforward.

6. Traceability

- All pipeline executions and errors must be logged via TraceBus and TraceSink.
- Logs must provide sufficient detail for debugging and performance analysis.

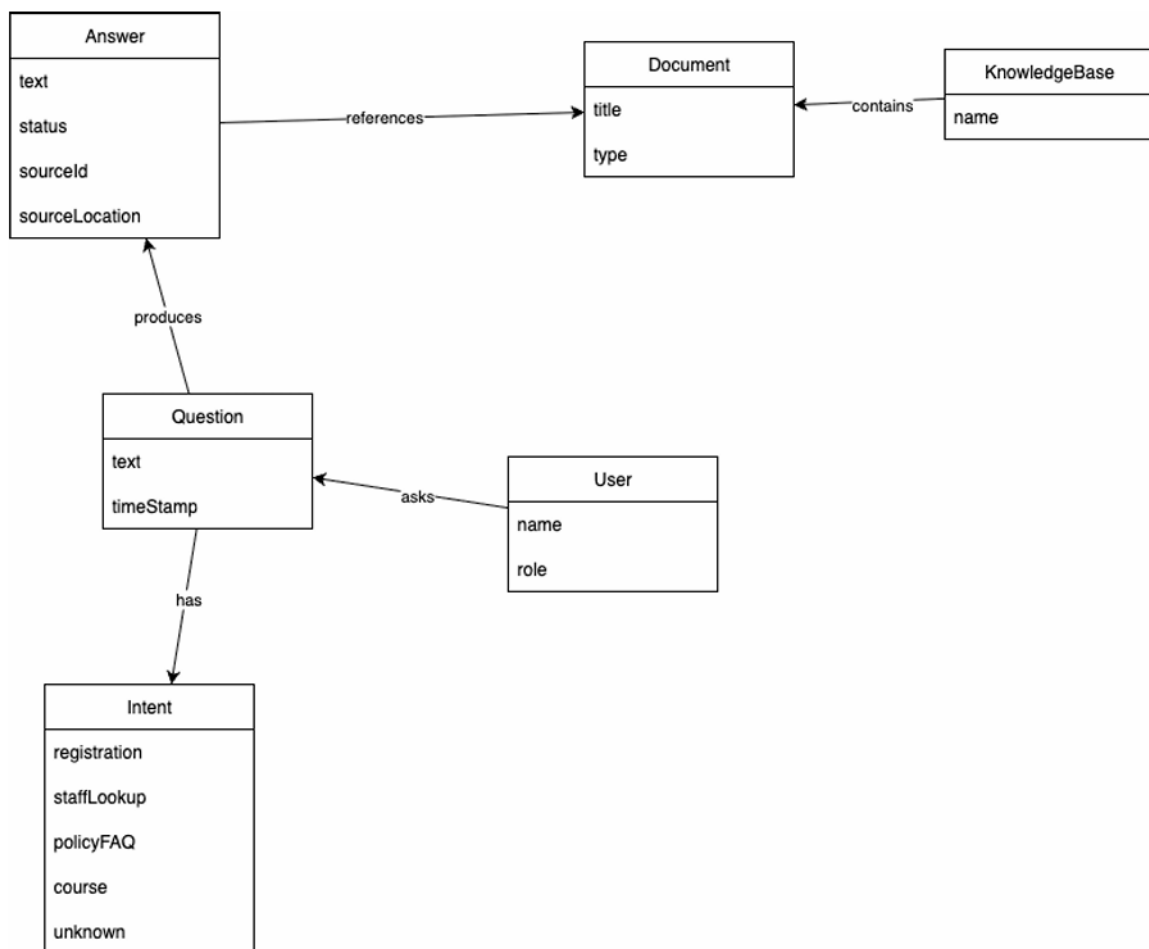
7. Portability

- The system must be platform-independent (Windows, Linux, macOS).
- Reliance on OS-specific features must be avoided (Java-based consistency).

8. Extensibility

- New components (Retrievers, Rerankers, Agents) must be integrable with minimal effort.
- The architecture must support plugin-like additions without modifying core logic.

Domain Model



Use Cases

Use Case 1 — Staff Information Lookup

Primary Actor

User (Student / Staff)

Goal

Retrieve accurate and up-to-date information about a specific university staff member

Preconditions

- The system is running and accessible through the CLI.
- Staff data has been indexed and is available in the document store.

Postconditions

- Relevant staff information is displayed to the user.

Trigger

The user enters a question about a staff member (e.g. “Who is Yusuf Ibrahim Matur?”).

Main Success Scenario

1. The user enters a staff-related question.
2. The system detects the user’s intent as “Staff Information Lookup.”
3. The system extracts keywords from the question.
4. The retrieval module searches the knowledge base for staff-related documents.
5. The system ranks the retrieved results based on relevance.
6. The answer generator summarizes the relevant information.
7. The system displays the staff information and the supporting citation.

Extensions / Alternative Flows

E1. No Relevant Document Found:

4a. The system cannot find a matching document.

4b. The fallback stage returns a response and system terminates

E2. Ambiguous Results:

5a. Multiple staff members match the query.

5b. The system returns the most probable match.

Special Requirements

- The system must respond within a reasonable time (< 5 seconds).
- Information must include citations to ensure reliability.

Use Case 2 — Course Information Query

Primary Actor

User (Student)

Goal

Obtain course details such as course content, prerequisites, credit hours, or instructor.

Preconditions

- Course descriptions and curriculum entries are available in the indexed dataset.

Postconditions

- The user receives structured and accurate course information.

Trigger

The user asks a course-related question (e.g. “cse3063 kaç kredi ve saat”).

Main Success Scenario

1. The user submits a course-related question.
2. The system classifies the intent as “Course Information Query.”
3. Query terms are generated from the user question.
4. The retrieval module searches course documents.
5. The reranking algorithm orders passages by relevance.
6. The answer generator constructs a final response.
7. The system returns the course information along with citations.

Extensions / Alternative Flows

E1. Course Code Missing or Partial:

3a. The system returns the most relevant result from matching other words.

E2. No Matching:

4a. The fallback system responds politely that no information could be found.

Special Requirements

- The system should support variations in user phrasing.
- Course codes should be recognized even when written in lowercase etc. (e.g., “cse3063”).

Use Case 3 — Policy/Regulation Query

Primary Actor

User (Student / Staff)

Goal

Retrieve university policies, rules, or regulations (e.g., attendance rules, grading policy, exam regulations).

Preconditions

- Policy documents are included in the dataset and have been processed and chunked.

Postconditions

- Relevant policy information is provided with citations.

Trigger

The user asks a policy-related question (e.g. “yaz okulu kaç haftadır?”).

Main Success Scenario

1. The user enters a question about a rule or policy.
2. The system identifies the intent as “Policy Query.”
3. The system generates related search terms.
4. The retriever locates matching policy sections.

5. The reranker prioritizes the most relevant passages.
6. The system formulates a concise, citation-backed answer.
7. The policy information is displayed.

Extensions / Alternative Flows

E1. Conflicting or Multiple Policies Found:

- 5a. The system selects the most relevant chunk or indicates that results may differ.

E2. No Policy Document Found:

- 4a. The fallback system responds politely that no information could be found.

Special Requirements

- Policy answers must be accurate and citation-verified.
- The system must avoid generating unsupported or hallucinated rules.